Lập trình mạng · Chương 2

# Đa luồng: synchronized, happens-before và wait/notify

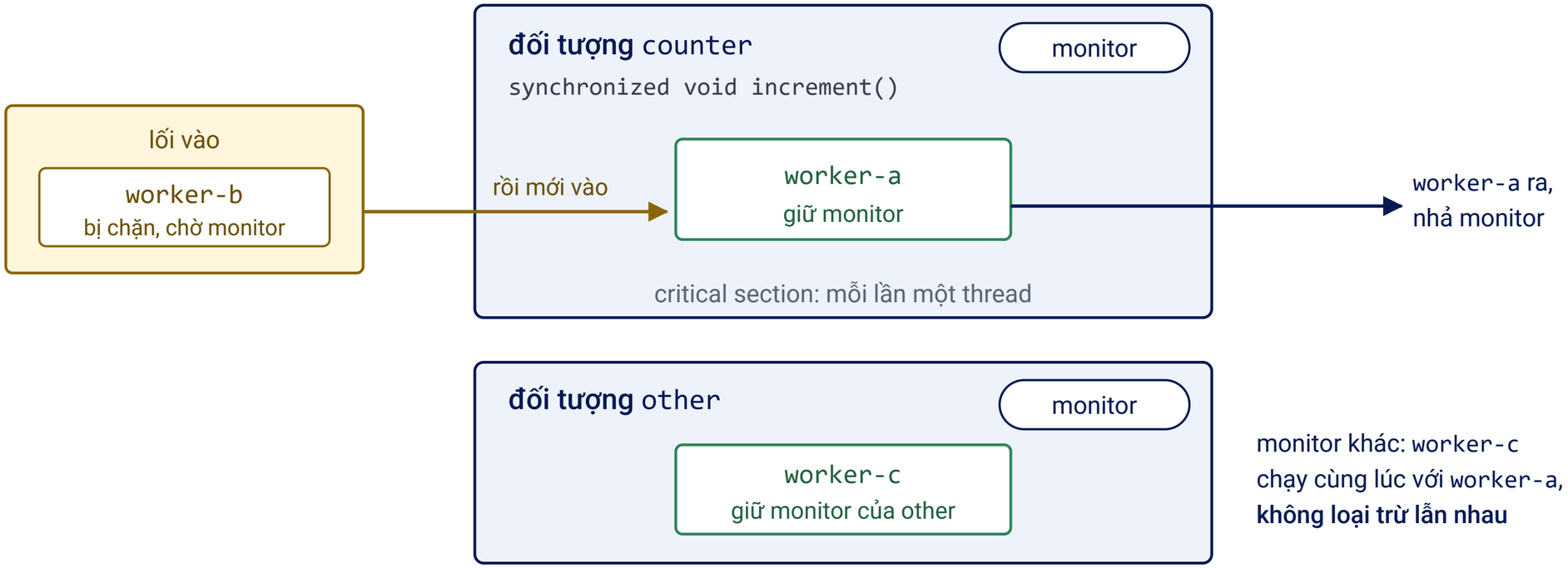
TS. Đặng Ngọc Hùng

# Dữ liệu dùng chung cần loại trừ, nhìn thấy được, và có cách chờ điều kiện

1. Bộ đếm `requests`: hai thread tăng 2 000 000 lần, **mất cập nhật**
2. Cần: **mỗi lần một thread** đọc – sửa – ghi, và giá trị đã ghi phải **nhìn thấy được** ở thread khác
3. Server đa luồng còn cần **chờ điều kiện**: thread xử lý chờ khi hàng đợi kết nối rỗng

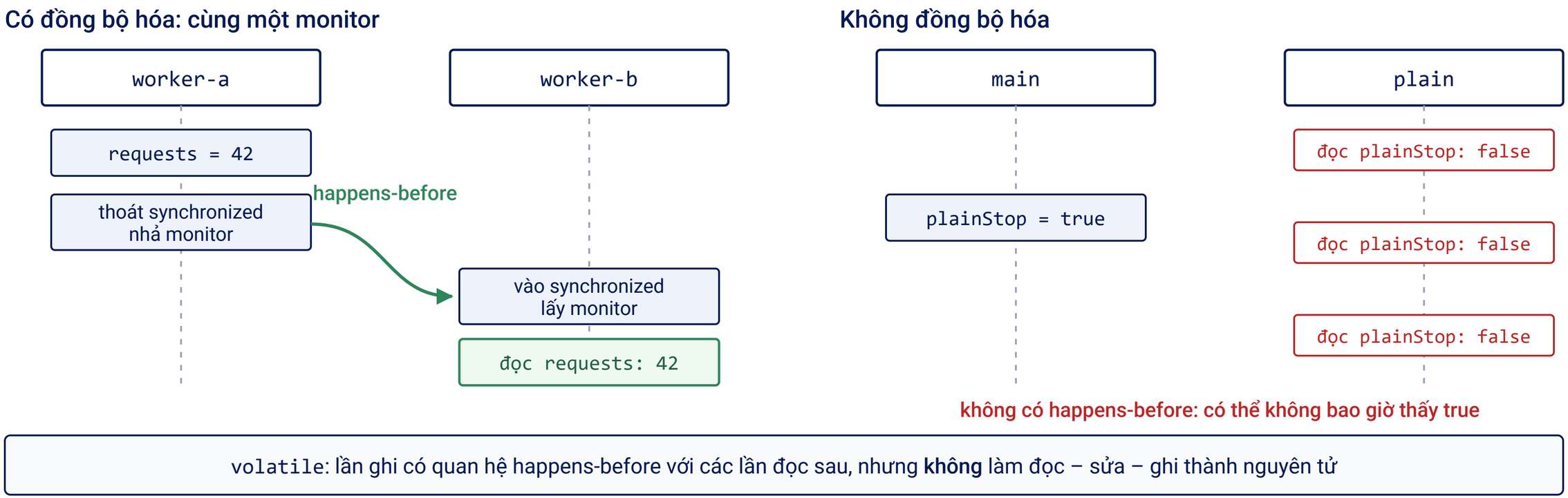
> **Câu hỏi của phần:** bảo vệ dữ liệu dùng chung bằng gì, bảo đảm nhìn thấy thay đổi ra sao, và chờ điều kiện thế nào cho đúng?

# synchronized cho mỗi lần một thread vào critical section



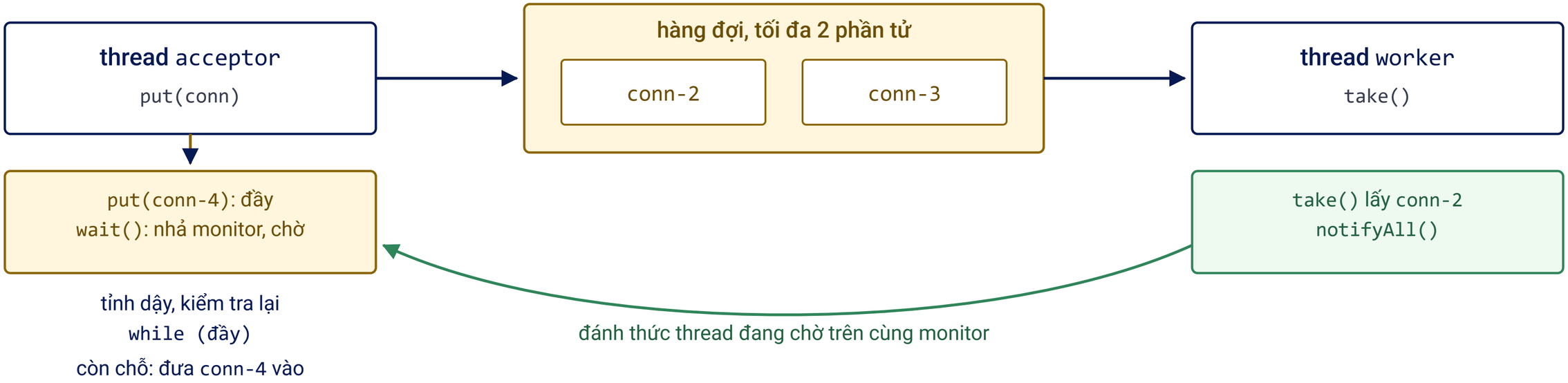

**synchronized** giữ **monitor** của đối tượng: trên cùng một đối tượng, mỗi lần một thread vào **critical section**; hai đối tượng khác nhau thì không loại trừ lẫn nhau.

# Nhả rồi lấy cùng một monitor tạo quan hệ happens-before




**Happens-before**: thay đổi trước khi nhả monitor nhìn thấy được sau khi lấy cùng monitor. Không đồng bộ hóa thì **không có bảo đảm** nào.

# wait() nhả monitor và chờ, notifyAll() đánh thức thread đang chờ




**wait/notify**: `wait()` nhả monitor và chờ; `notifyAll()` đánh thức; thread tỉnh dậy **kiểm tra lại** điều kiện trong `while`.

# Bộ đếm có synchronized cho đúng 2 000 000 ở mọi lần chạy

Java

```
synchronized void increment() {  // critical section: mỗi lần một thread, giữ monitor của this
    requests++;
}

synchronized int get() {  // đọc cũng giữ cùng monitor: thấy giá trị mới nhất
    return requests;
}
```

Kết quả chạy

```
run 1: requests = 2000000 (expected 2000000)
run 2: requests = 2000000 (expected 2000000)
run 3: requests = 2000000 (expected 2000000)
```

# Hàng đợi đầy thì put() chờ đến khi worker lấy bớt

Java

```
synchronized void put(T item) throws InterruptedException {
    while (items.size() == capacity) {  // đầy: chờ; wait() nhả monitor trong lúc chờ
        wait();
    }
    items.addLast(item);
    notifyAll();  // đánh thức các thread đang chờ trên monitor này
}
```

Kết quả chạy

```
    9 ms  acceptor  put  conn-1
    9 ms  worker    took conn-1
   16 ms  acceptor  put  conn-2
   16 ms  acceptor  put  conn-3
  318 ms  worker    took conn-2
  318 ms  acceptor  put  conn-4  (waited 301 ms: buffer full)
  621 ms  worker    took conn-3
  621 ms  acceptor  put  conn-5  (waited 298 ms: buffer full)
  922 ms  worker    took conn-4
 1237 ms  worker    took conn-5
```

# wait() phải nằm trong while: thread tỉnh dậy chưa chắc điều kiện đã đúng

Java

```
synchronized String take() throws InterruptedException {
    if (items.isEmpty()) {  // SAI: chỉ kiểm tra điều kiện một lần
        wait();
    }
    return items.removeFirst();  // tỉnh dậy mà hàng đợi vẫn rỗng thì ném ngoại lệ
}
```

Kết quả chạy

```
worker-1: took conn-1
worker-2: NoSuchElementException
```

- `notifyAll()` đánh thức **cả hai** worker, nhưng chỉ có **một** phần tử
- Thread còn có thể tỉnh dậy **không do ai đánh thức** (spurious wakeup, đánh thức giả): luôn kiểm tra lại điều kiện trong `while`

# Câu hỏi 1

## Vì sao lời gọi `wait()` trong `take()` phải nằm trong vòng lặp while thay vì if?

A. Vì Java chỉ cho phép gọi `wait()` bên trong một vòng lặp.

B. Vì vòng lặp while chạy nhanh hơn câu lệnh if.

C. Vì thread tỉnh dậy chưa chắc điều kiện đã đúng: thread khác có thể đã lấy mất phần tử, hoặc thread tỉnh dậy không do ai đánh thức.

D. Vì `notifyAll()` chỉ đánh thức được thread đang chờ trong vòng lặp.

# Câu hỏi 1

## Vì sao lời gọi `wait()` trong `take()` phải nằm trong vòng lặp while thay vì if?

A. Vì Java chỉ cho phép gọi `wait()` bên trong một vòng lặp.

B. Vì vòng lặp while chạy nhanh hơn câu lệnh if.

**C. Vì thread tỉnh dậy chưa chắc điều kiện đã đúng: thread khác có thể đã lấy mất phần tử, hoặc thread tỉnh dậy không do ai đánh thức.**

D. Vì `notifyAll()` chỉ đánh thức được thread đang chờ trong vòng lặp.

`notifyAll()` đánh thức mọi thread đang chờ, và thread có thể tỉnh dậy do spurious wakeup. Thread tỉnh dậy phải kiểm tra lại điều kiện. Dùng if vẫn biên dịch được, chỉ sai khi chạy.

# Câu hỏi 2

**Hai thread cùng lúc gọi phương thức synchronized `increment()`: một thread trên đối tượng counter, thread kia trên đối tượng other cùng lớp. Điều nào đúng?**

A. Hai thread chạy cùng lúc được, vì mỗi đối tượng có monitor riêng.

B. Thread thứ hai phải chờ, vì synchronized khóa phương thức `increment()` cho mọi đối tượng.

C. Chương trình báo lỗi vì hai thread cùng gọi một phương thức synchronized.

D. Thread thứ hai phải chờ, vì mọi phương thức synchronized trong chương trình dùng chung một khóa.

# Câu hỏi 2

## Hai thread cùng lúc gọi phương thức synchronized `increment()`: một thread trên đối tượng counter, thread kia trên đối tượng other cùng lớp. Điều nào đúng?

A. Hai thread chạy cùng lúc được, vì mỗi đối tượng có monitor riêng.

B. Thread thứ hai phải chờ, vì synchronized khóa phương thức `increment()` cho mọi đối tượng.

C. Chương trình báo lỗi vì hai thread cùng gọi một phương thức synchronized.

D. Thread thứ hai phải chờ, vì mọi phương thức synchronized trong chương trình dùng chung một khóa.

Phương thức synchronized (không static) giữ monitor của đối tượng đang gọi. counter và other là hai đối tượng, hai monitor, nên hai thread không loại trừ nhau.

# Tổng kết

1. **synchronized + monitor**: trên cùng một đối tượng, mỗi lần một thread vào **critical section**
2. **Happens-before**: nhả rồi lấy cùng monitor, hoặc ghi rồi đọc biến **volatile**, thì thay đổi nhìn thấy được
3. **wait/notify**: `wait()` nhả monitor và chờ, `notifyAll()` đánh thức; điều kiện kiểm tra trong `while`

**Từ khóa cần nhớ** critical section · synchronized · monitor · happens-before · wait/notify · volatile

# Mở rộng

### Chủ đề nâng cao

- java.util.concurrent: ReentrantLock, Condition, BlockingQueue
- Deadlock và thứ tự lấy khóa

### Bài tập mở rộng

Thay BoundedBuffer tự viết bằng ArrayBlockingQueue; so sánh số dòng mã và hành vi khi hàng đợi đầy.

Tiếp theo

# Đa luồng: Executor, thread pool và virtual thread